



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа по дисциплине «Операционные системы»

Тема Буферизованный и небуферизованный ввод-вывод

Студент Палладий Е. И.

Группа ИУ7-61Б

Преподаватели Рязанова Н. Ю.

Москва, 2026

1 Структуры

1.1 FILE и структура _IO_FILE

Листинг 1.1 — FILE /libio/bits/types/FILE.h

```
#ifndef __FILE_defined
#define __FILE_defined 1
struct _IO_FILE;
/* The opaque type of streams. This is the definition used elsewhere. */
typedef struct _IO_FILE FILE;
#endif
```

Листинг 1.2 — _IO_FILE /libio/bits/types/struct_FILE.h

```
struct _IO_FILE
{
    int _flags; /* High-order word is _IO_MAGIC; rest is flags. */

    /* The following pointers correspond to the C++ streambuf protocol. */
    char *_IO_read_ptr; /* Current read pointer */
    char *_IO_read_end; /* End of get area. */
    char *_IO_read_base; /* Start of putback+get area. */
    char *_IO_write_base; /* Start of put area. */
    char *_IO_write_ptr; /* Current put pointer. */
    char *_IO_write_end; /* End of put area. */
    char *_IO_buf_base; /* Start of reserve area. */
    char *_IO_buf_end; /* End of reserve area. */

    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base; /* Pointer to start of non-current get area. */
    char *_IO_backup_base; /* Pointer to first valid character of backup area */
    char *_IO_save_end; /* Pointer to end of non-current get area. */

    struct _IO_marker *_markers;

    struct _IO_FILE *_chain;

    int _fileno;
    int _flags2:24;
    /* Fallback buffer to use when malloc fails to allocate one. */
```

```

char _short_backupbuf[1];
__off_t _old_offset; /* This used to be _offset but it's too small. */

/* 1+column number of pbase(); 0 is unknown. */
unsigned short _cur_column;
signed char _vtable_offset;
char _shortbuf[1];

_IO_lock_t *_lock;
#ifdef _IO_USE_OLD_IO_FILE
};

struct _IO_FILE_complete
{
    struct _IO_FILE _file;
#ifdef
    __off64_t _offset;
    /* Wide character stream stuff. */
    struct _IO_codecvt *_codecvt;
    struct _IO_wide_data *_wide_data;
    struct _IO_FILE *_freeres_list;
    void *_freeres_buf;
    struct _IO_FILE **_prevchain;
    int _mode;
    #if __WORDSIZE == 64
    int _unused3;
    #endif
    __uint64_t _total_written;
    #if __WORDSIZE == 32
    int _unused3;
    #endif
    /* Make sure we don't get into trouble again. */
    char _unused2[12 * sizeof (int) - 5 * sizeof (void *)];
};

```

1.2 Структура stat

Листинг 1.3 — struct stat

```

struct stat {
    dev_t      st_dev;      /* устройство */

```

```

ino_t      st_ino;      /* inode */
mode_t     st_mode;     /* режим доступа */
nlink_t    st_nlink;    /* количество жестких ссылок */
uid_t      st_uid;      /* идентификатор пользователя-владельца */
gid_t      st_gid;      /* идентификатор группы-владельца */
dev_t      st_rdev;     /* тип устройства (если это устройство)*/
off_t      st_size;     /* общий размер в байтах */
blksize_t  st_blksize;  /* размер блока ввода-вывода в файловой системе */
blkcnt_t   st_blocks;   /* количество выделенных 512-байтных блоков */
union { time_t st_atime; struct timespec st_atim; }; /* время последнего доступа */
union { time_t st_mtime; struct timespec st_mtim; }; /* время последней модификации */
union { time_t st_ctime; struct timespec st_ctim; }; /* время последнего изменения */
};

```

2 Программа №1

2.1 Код программы

Листинг 2.1 — Код первой программы

```
#include <stdio.h>
#include <fcntl.h>

int main()
{
    int fd = open("alphabet.txt", O_RDONLY);

    FILE *fs1 = fdopen(fd, "r");
    char buff1[20];
    setvbuf(fs1, buff1, _IOFBF, 20);

    FILE *fs2 = fdopen(fd, "r");
    char buff2[20];
    setvbuf(fs2, buff2, _IOFBF, 20);

    int flag1 = 1, flag2 = 2;
    while(flag1 == 1 || flag2 == 1)
    {
        char c;
        flag1 = fscanf(fs1, "%c", &c);
        if (flag1 == 1)
        {
            fprintf(stdout, "%c", c);
        }
        flag2 = fscanf(fs2, "%c", &c);
        if (flag2 == 1)
        {
            fprintf(stdout, "%c", c);
        }
    }
    return 0;
}
```

2.2 Результат работы программы

```
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ gcc p1.c
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
aubvcwdxeyfzghijklmnopqrstwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$
```

Рисунок 2.1 — Результат работы первой программы

2.3 Анализ работы программы

Системный вызов `open` открывает файл `alphabet.txt` в режиме «только для чтения», создаёт структуру `file`. Ссылка на эту структуру помещается в таблицу файловых дескрипторов процесса. Поскольку дескрипторы 0, 1 и 2 заняты стандартными потоками ввода-вывода, системный вызов возвращает наименьший свободный дескриптор (3), который присваивается переменной `fd`.

Библиотечной функции `fdopen` передается дескриптор `fd` и режим доступа «только для чтения». Функция возвращает указатель на инициализированную структуру `FILE`, где полю `_fileno` присваивается значение `fd`.

Функция `setvbuf` задаёт размер буфера 20 байт для `fs1` и `fs2`.

Функция `fscanf` для `fs1` обнаруживает, что буфер пуст, и выполняет системный вызов `read`, который считывает 20 символов («a»—«t») из файла «alphabet.txt» в буфер `buff1`. При этом значение поля `f_pos` структуры `file` в ядре увеличивается на 20. Затем из буфера извлекается первый символ («a»), записывается в переменную `s`, а указатель `_IO_read_ptr` в структуре `_IO_FILE` сдвигается на 1 байт. Вызов `fprintf` выводит этот символ в стандартный поток вывода.

Так как структуры `_IO_FILE` `fs1` и `fs2` содержат одинаковое значение поля `_fileno` и, следовательно, ссылаются на одну и ту же структуру `file` в ядре, где `f_pos` уже равно 20, то при вызове `fscanf` для `fs2` системный вызов `read` считывает оставшиеся 6 символов («u»—«z») в буфер `buff2`. Первый символ из буфера `fs2` («u») записывается в переменную `s` и выводится в стандартный поток вывода.

В последующих итерациях цикла поочерёдно будут выводиться символы из буферов `fs1` и `fs2`. Когда будет достигнут конец буфера `fs2` (сравняются указатели `_IO_read_ptr` и `_IO_read_end`), то программа продолжит выводить оставшиеся символы только из буфера `fs1` (вплоть до `t`), пока не будет достигнут конец буфера `fs1`.

2.4 Иллюстрация связи структур

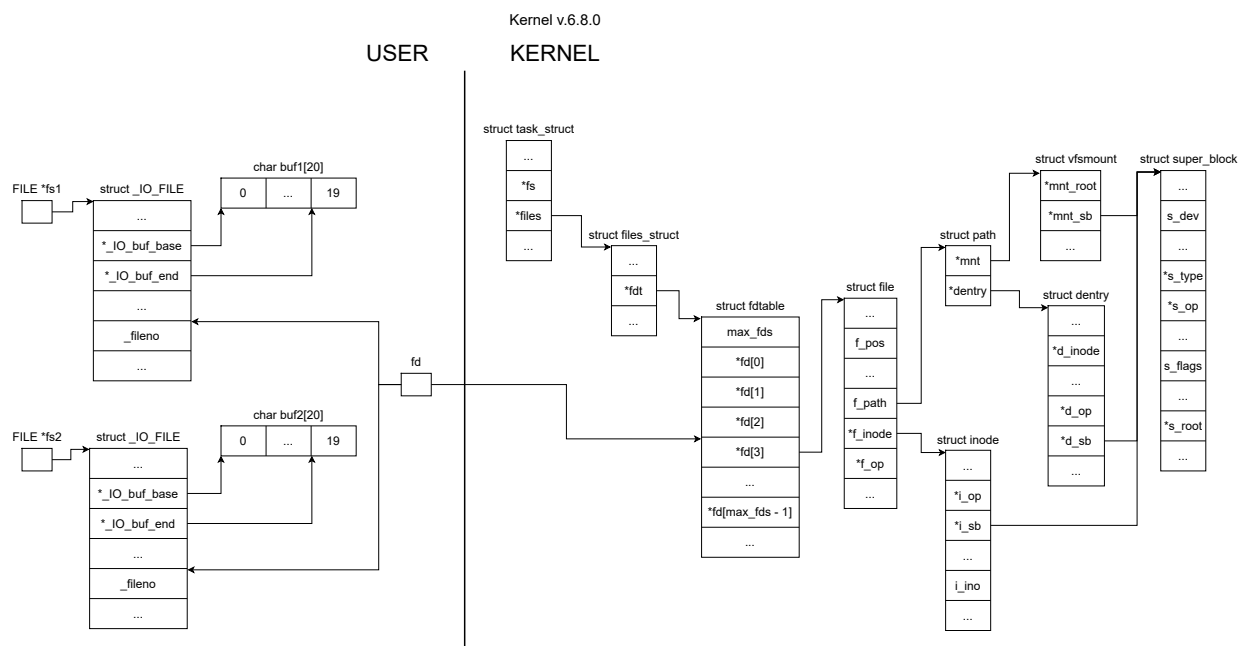


Рисунок 2.2 — Иллюстрация связи структур в первой программе

3 Программа №2

3.1 Однопоточный вариант программы

3.1.1 Код программы

Листинг 3.1 — Код однопоточной первой программы

```
#include <fcntl.h>
#include <unistd.h>

int main()
{
    char c;
    int fl1 = 1, fl2 = 1;

    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);

    while((fl1 == 1) && (fl2 == 1))
    {
        if ((fl1 = read(fd1, &c, 1)) == 1)
        {
            write(1, &c, 1);
        }

        if (fl1 == 1 && ((fl2 = read(fd2, &c, 1)) == 1))
        {
            write(1, &c, 1);
        }
    }
    return 0;
}
```


3.1.2 Результат работы программы

```
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ gcc p2_read_1thread.c
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
aabbccddeeffgghhiijjkkllmmnnnooppqqrrssttuuvvwxxyyzzwhxeqh@ubuntu:~/bmstu/sem_06/OS/B
```

Рисунок 3.1 — Результат работы однопоточной второй программы

3.1.3 Анализ работы программы

Системный вызов `open` создаёт два дескриптора открытого файла в режиме «только для чтения» (`fd1`, `fd2`). Соответствующие дескрипторам структуры `file` ссылаются на один и тот же `inode` файла «`alphabet.txt`». Ссылки на эти структуры помещаются в таблицу открытых процессом файлов под индексами 3 и 4, так как дескрипторы 0,1 и 2 заняты стандартными потоками.

Системный вызов `read` для дескриптора `fd1` считывает один символ («а»), что приводит к увеличению на 1 значения поля `f_pos` в соответствующей структуре `file`. Считанный символ выводится в стандартный поток вывода.

Так как дескрипторам `fd1` и `fd2` соответствуют независимые структуры `file`, то для `fd2` значение поля `f_pos` равно 0. Поэтому при вызове `read` для `fd2` будет считан тот же самый первый символ («а»), который также выведется в стандартный поток вывода.

Поочерёдное чтение и вывод символов будут продолжаться в цикле, пока значения `f_pos` в обеих структурах `file` не достигнут конца файла (размера файла, хранящегося в поле `i_size` и равного 26 байтам).

Таким образом, в стандартный поток вывода каждый символ алфавита будет выведен строго последовательно дважды.

3.2 Вариант программы с 1 дополнительным потоком

3.2.1 Код программы

Листинг 3.2 — Код программы с 1 дополнительным потоком

```
#include <fcntl.h>
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

void *thread_func(void *arg)
{
    char c;
```

```

int fl = 1;
int fd = *((int *)arg);

while (fl == 1)
{
    if ((fl = read(fd, &c, 1)))
        write(1, &c, 1);
}
return NULL;
}

int main()
{
    pthread_t t;
    char c;
    int fl = 1;

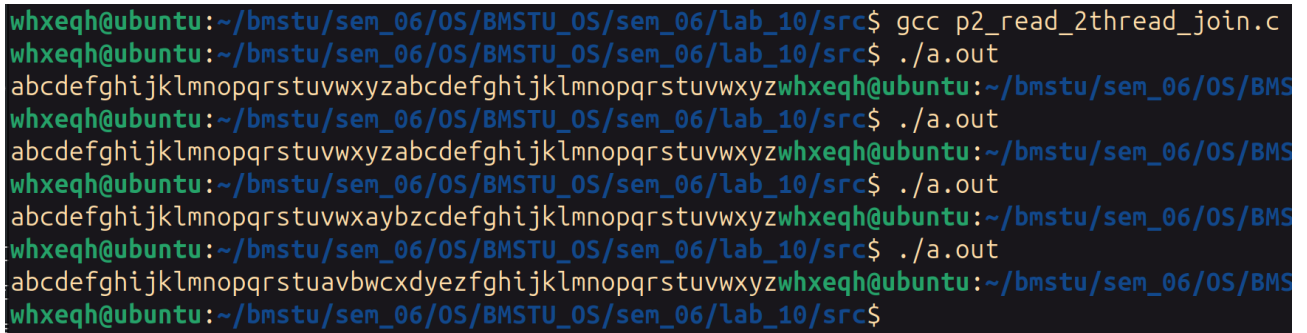
    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);

    if (pthread_create(&t, NULL, thread_func, &fd1) != 0)
    {
        perror("pthread_create");
        exit(1);
    }

    while (fl == 1)
    {
        if ((fl = read(fd2, &c, 1)) == 1)
            write(1, &c, 1);
    }
    pthread_join(t, NULL);
    return 0;
}

```

3.2.2 Результат работы программы



```
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ gcc p2_read_2thread_join.c
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
abcdefghijklmnopqrstuvwxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
abcdefghijklmnopqrstuvwxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
abcdefghijklmnopqrstuvwxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
abcdefghijklmnopqrstuvwxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
abcdefghijklmnopqrstuvwxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$
```

Рисунок 3.2 — Результат работы программы с 1 дополнительным потоком

3.2.3 Анализ работы программы

Вызов `pthread_create` создаёт дополнительный поток, которому в качестве аргумента передаётся указатель на дескриптор `fd1`. Поскольку на создание потока требуется время, главный поток может успеть прочитать и вывести либо весь алфавит, либо его часть.

После создания дополнительного потока чтение и вывод символов осуществляются обоими потоками в зависящей от планировщика последовательности.

Когда главный поток завершит чтение по своему дескриптору, он выйдет из цикла и заблокируется в вызове `pthread_join`, ожидая завершения дополнительного потока. Дополнительный поток продолжит работу и выведет оставшуюся часть своих символов.

Таким образом, каждый символ файла будет напечатан дважды, однако точный порядок их вывода не определён и полностью зависит от того, как планировщик распределяет процессорное время между потоками.

3.3 Вариант программы с 2 дополнительными потоками

3.3.1 Код программы

Листинг 3.3 — Код программы с 2 дополнительными потоками

```
#include <fcntl.h>
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

void *thread_func(void *arg)
{
    char c;
```

```

int fl = 1;
int fd = *((int *)arg);

while (fl == 1)
{
    if ((fl = read(fd, &c, 1)))
        write(1, &c, 1);
}
return NULL;
}

int main()
{
    pthread_t t1, t2;

    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);

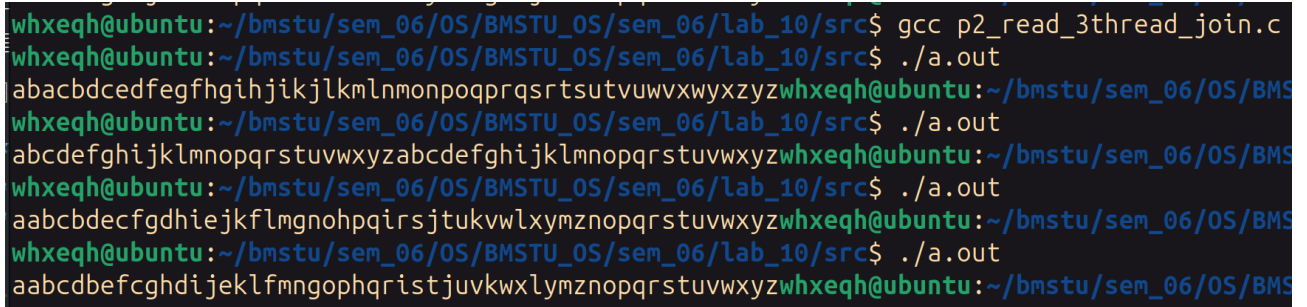
    if (pthread_create(&t1, NULL, thread_func, &fd1) != 0)
    {
        perror("pthread_create");
        exit(1);
    }
    if (pthread_create(&t2, NULL, thread_func, &fd2) != 0)
    {
        perror("pthread_create");
        exit(1);
    }

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

3.3.2 Результат работы программы



```
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ gcc p2_read_3thread_join.c
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
abacbdcedfegfhghijikjklmnlmonpoqprqsrtstuvvwxyxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
aabcdbdecfgdhiejkflmgnohpqirsjtuksvwlymznopqrstuvwxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
aabcdbefcgdhiejkflmgnohpqirsjtuksvwlymznopqrstuvwxyzwhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$
```

Рисунок 3.3 — Результат работы программы с 2 дополнительными потоками

3.3.3 Анализ работы программы

Два вызова `pthread_create` создают два дополнительных потока. Каждому из них передаётся указатель на свой файловый дескриптор (первому — `fd1`, второму — `fd2`). Главный поток, создав рабочие потоки, сразу же блокируется в вызовах `pthread_join`, ожидая их завершения.

На создание потоков требуется время и их порядок выполнения зависит от планировщика, поэтому порядок вывода символов неизвестен.

Таким образом, каждый символ файла будет напечатан дважды, однако точный порядок их вывода не определён и полностью зависит от того, как планировщик распределяет процессорное время между дополнительными потоками.

3.4 Иллюстрация связи структур

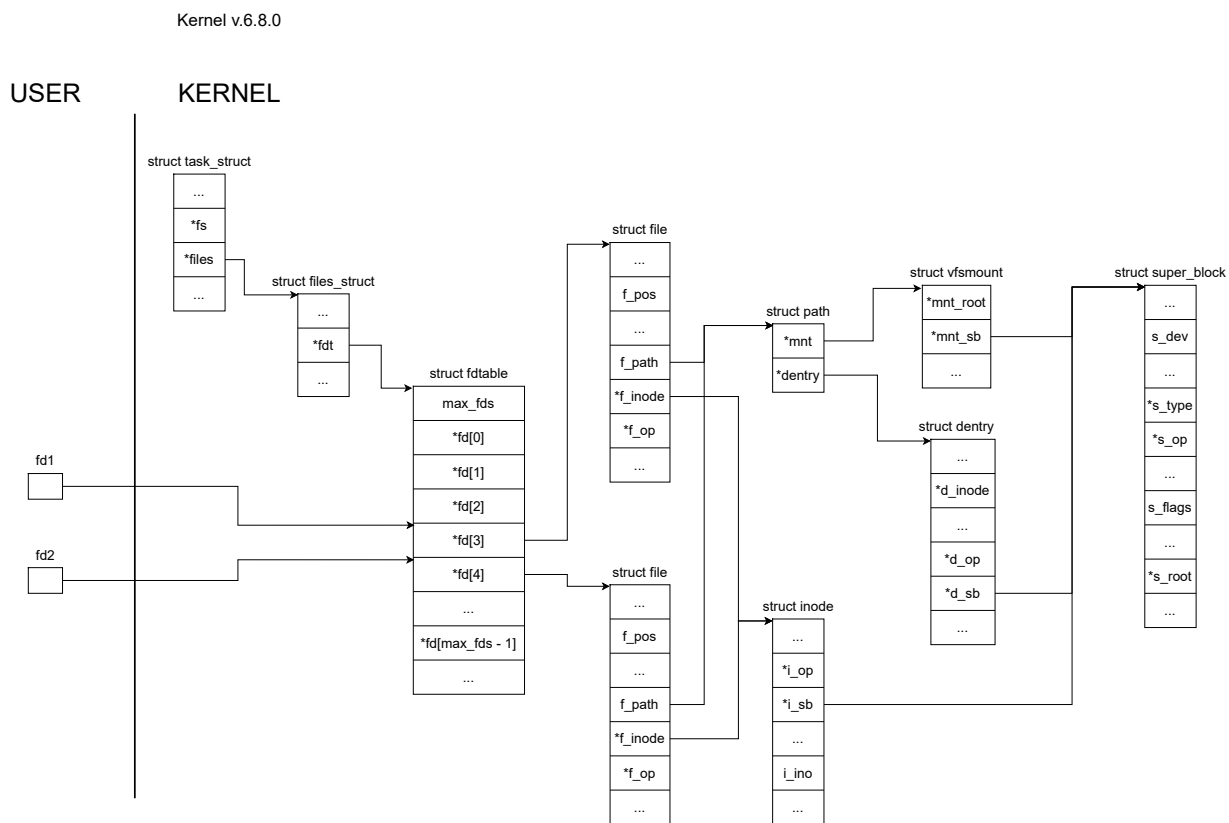


Рисунок 3.4 — Иллюстрация связи структур во второй программе

4 Программа №3

4.1 Вариант программы с open, write и close

4.1.1 Код программы

Листинг 4.1 — Код программы с open, write и close

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>

int main()
{
    struct stat statbuf;
    int fd1 = open("q.txt", O_RDWR);
    int fd2 = open("q.txt", O_RDWR);
    stat("q.txt", &statbuf);
    printf("inode: %lu;size: %ld; blksize: %ld\n", statbuf.st_ino, statbuf.st_size,
    statbuf.st_blksize);

    for(char c = 'a'; c <= 'z'; c++)
    {
        if (c%2)
            write(fd1, &c, 1);
        else
            write(fd2, &c, 1);
        stat("q.txt", &statbuf);
        printf("inode: %lu;size: %ld; blksize: %ld\n", statbuf.st_ino,
        statbuf.st_size, statbuf.st_blksize);
    }
    close(fd1);
    close(fd2);
    return 0;
}
```

4.1.2 Результат работы программы

```
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ gcc p3_write.c
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
inode: 22439113;size: 0; blksize: 4096
inode: 22439113;size: 1; blksize: 4096
inode: 22439113;size: 1; blksize: 4096
inode: 22439113;size: 2; blksize: 4096
inode: 22439113;size: 2; blksize: 4096
inode: 22439113;size: 3; blksize: 4096
inode: 22439113;size: 3; blksize: 4096
inode: 22439113;size: 4; blksize: 4096
inode: 22439113;size: 4; blksize: 4096
inode: 22439113;size: 5; blksize: 4096
inode: 22439113;size: 5; blksize: 4096
inode: 22439113;size: 6; blksize: 4096
inode: 22439113;size: 6; blksize: 4096
inode: 22439113;size: 7; blksize: 4096
inode: 22439113;size: 7; blksize: 4096
inode: 22439113;size: 8; blksize: 4096
inode: 22439113;size: 8; blksize: 4096
inode: 22439113;size: 9; blksize: 4096
inode: 22439113;size: 9; blksize: 4096
inode: 22439113;size: 10; blksize: 4096
inode: 22439113;size: 10; blksize: 4096
inode: 22439113;size: 11; blksize: 4096
inode: 22439113;size: 11; blksize: 4096
inode: 22439113;size: 12; blksize: 4096
inode: 22439113;size: 12; blksize: 4096
inode: 22439113;size: 13; blksize: 4096
inode: 22439113;size: 13; blksize: 4096
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ cat q.txt
bdfhjlnprtvxzw
```

Рисунок 4.1 — Результат работы программы с open, write и close

4.2 Анализ работы программы

Системный вызов `open` создаёт два дескриптора открытого файла в режиме «чтения и записи» (`fd1`, `fd2`). Соответствующие дескрипторам структуры `file` ссылаются на один и тот же `inode` файла «`q.txt`». Ссылки на эти структуры помещаются в таблицу открытых процессом файлов под индексами 3 и 4, так как дескрипторы 0, 1 и 2 заняты стандартными потоками.

В первой итерации цикла системный вызов `write` для дескриптора `fd1` записывает символ «а» в открытый файл, что приводит к увеличению на 1 значения поля `f_pos` в соответствующей структуре `file`. Размер файла становится равным 1 байту.

Так как дескрипторам `fd1` и `fd2` соответствуют независимые структуры `file`, то для `fd2` значение поля `f_pos` остаётся равным 0. Поэтому на второй итерации цикла вызов `write` для `fd2`

заменит символ «a» в файле на символ «b». Размер файла остаётся равным 1 байту, а значение `f_pos` для `fd2` увеличится на 1.

Поочерёдная запись символов в файл продолжается до конца цикла.

Таким образом, в файл будет записана только каждая вторая буква алфавита, начиная с «b», и итоговый размер файла составит 13 байт, так как вызов `write` для `fd2` будет заменять символы, записанные вызовом `write` для `fd1`.

4.3 Иллюстрация связи структур

Иллюстрация связи структур представлена на рисунке ??.

4.4 Вариант программы с `fopen`, `fprintf` и `fclose`

4.4.1 Код программы

Листинг 4.2 — Код программы с `fopen`, `fprintf` и `fclose`

```
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>

int main()
{
    struct stat statbuf;
    FILE *fs1 = fopen("q.txt", "w");
    FILE *fs2 = fopen("q.txt", "w");
    stat("q.txt", &statbuf);
    printf("inode: %lu;size: %ld; blksize: %ld\n", statbuf.st_ino, statbuf.st_size,
        statbuf.st_blksize);

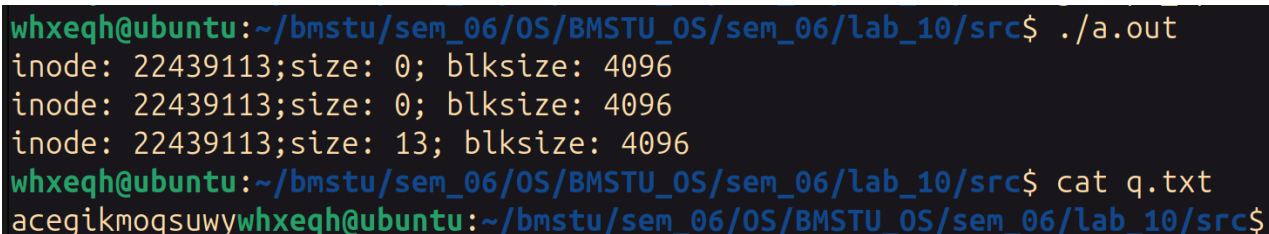
    for(char c = 'a'; c <= 'z'; c++)
    {
        if (c%2)
            fprintf(fs1, "%c", c);
        else
            fprintf(fs2, "%c", c);
    }
    stat("q.txt", &statbuf);
    printf("inode: %lu;size: %ld; blksize: %ld\n", statbuf.st_ino, statbuf.st_size,
        statbuf.st_blksize);
    fclose(fs2);
}
```

```

fclose(fs1);
stat("q.txt", &statbuf);
printf("inode: %lu;size: %ld; blksize: %ld\n", statbuf.st_ino, statbuf.st_size,
statbuf.st_blksize);
return 0;
}

```

4.4.2 Результат работы программы



```

whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ ./a.out
inode: 22439113;size: 0; blksize: 4096
inode: 22439113;size: 0; blksize: 4096
inode: 22439113;size: 13; blksize: 4096
whxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$ cat q.txt
acegikmoqsuwywhxeqh@ubuntu:~/bmstu/sem_06/OS/BMSTU_OS/sem_06/lab_10/src$

```

Рисунок 4.2 — Результат работы программы с `fopen`, `fprintf` и `fclose`

4.4.3 Анализ работы программы

Вызов библиотечной функции `fopen` откроет файл «q.txt» в режиме «только для записи с усечением» и вернёт два указателя на инициализированные структуры `FILE` (`fs1`, `fs2`). Во время вызова создаются две независимые структуры `file` с дескрипторами 3 и 4, значения которых будут записаны в поля `_fileno` соответствующих структур `_IO_FILE`. Эти структуры `file` ссылаются на один и тот же inode файла «q.txt».

Вызов библиотечной функции `fprintf` для `fs1` записывает символ «а» в буфер, что приводит к смещению в соответствующей структуре указателя `_IO_WRITE_PTR` на 1 байт.

Вызов библиотечной функции `fprintf` для `fs2` записывает символ «b» в буфер, что приводит к смещению в соответствующей структуре указателя `_IO_WRITE_PTR` на 1 байт.

В результате работы цикла в буфере `fs1` будет находиться каждая вторая буква алфавита, начиная с «а», а в `fs2` — каждая вторая буква алфавита, начиная с «b».

Вызов `fclose` для `fs2` приводит к записи из буфера в открытый файл. Значение поля `f_pos` для дескриптора 4 и размера файла становится 13. После этого дескриптор закрывается.

Так как были созданы две структуры открытых файлов, ссылающихся на один и тот же inode, а в структуре, соответствующей `fs1`, значение поля `f_pos` осталось равным 0, то при вызове `fclose` для `fs1` буфер записывается в открытый файл с начала, что приводит к замене последовательности символов буфера `fs2` на последовательность символов буфера `fs1`. Размер файла остаётся неизменным (13 байт).

Таким образом, итоговое содержимое файла зависит от порядка вызовов `fclose`. В данном

случае в файл будет записан каждый второй символ алфавита, начиная с буквы «а», а размер файла составит 13 байт.

4.4.4 Иллюстрация связи структур

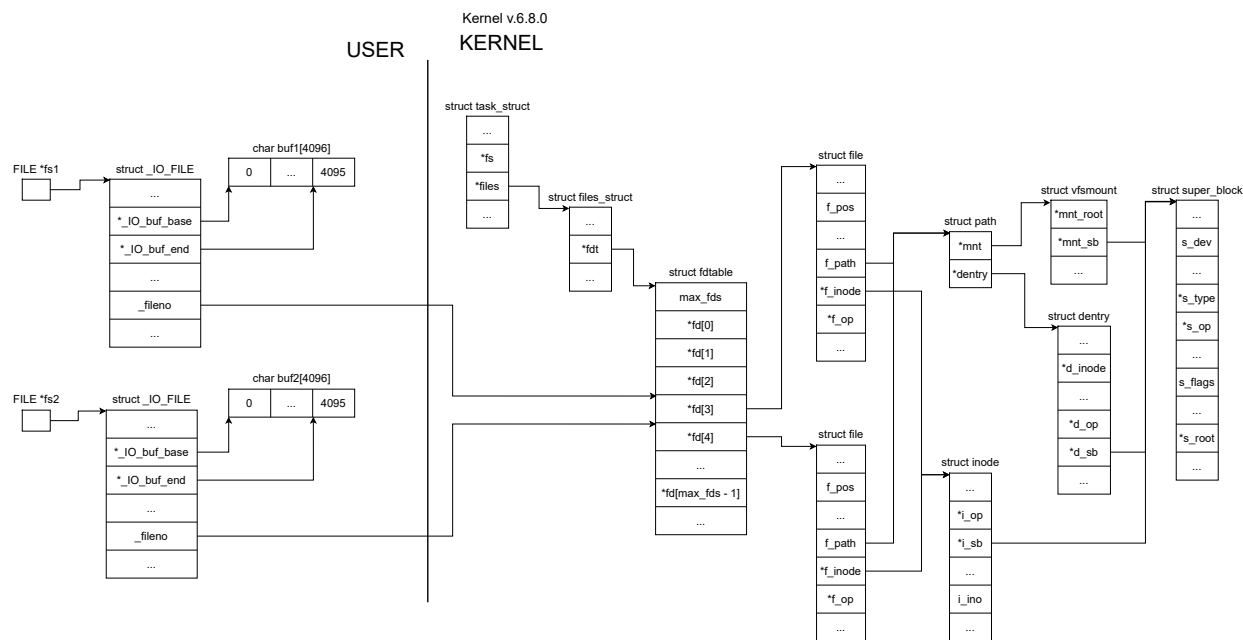


Рисунок 4.3 — Иллюстрация связи структур в третьей программе с `open`, `fprintf` и `fclose`